

Recolha, Análise e ***Tokenização de um Corpus***

Diogo Silva

Índice

Resumo.....	3
Introdução.....	4
Descrição dos dados recolhidos.....	5
Medidas identificadas.....	5
Aspetos relevantes identificados.....	6
Descrição das Tarefas Desenvolvidas e Procedimentos Efetuados.....	9
Análise do corpus.....	9
Criação e treino do tokenizador BPE.....	10
Utilização de tokenizadores pré-treinados.....	11
Conclusões.....	12
Bibliografia.....	13

Resumo

Este trabalho consiste na recolha, análise e tokenização de um corpus textual que foi elaborado a partir de excertos de obras de António Lobo Antunes, tendo sido posteriormente segmentado em conjuntos de treino (90%) e teste (10%).

Para a análise estatística e linguística do corpus, foi utilizado a biblioteca NLTK, permitindo calcular métricas como o número total de palavras, vocabulário, riqueza lexical, as palavras mais comuns e a sua frequência, a identificação de palavras estrangeiras, a cobertura do vocabulário do conjunto de teste, as palavras mais raras (Hapax legomenon) e a complexidade lexical (Hapax ratio). Foram também identificadas palavras fora do vocabulário (OOV) do conjunto de treino no conjunto de teste, utilizando os espaços em branco como divisão de tokens.

Por fim, foi implementado o método Byte Pair Encoding (BPE) para tokenização, sendo treinado no conjunto de treino e aplicado ao conjunto de teste. Os resultados foram comparados com métodos tradicionais e tokenizadores pré-treinados baseado em BPE e WordPiece.

Conclui-se que tokenizadores treinados com um corpus reduzido (6833 palavras) apresentam vantagens na segmentação de palavras, mas apresenta um percentual de OOV maior, criando assim inconsistências, já os modelos pré-treinados multilíngues, por serem treinados com um corpus muito mais extenso, oferecem robustez generalizada.

Introdução

Com o aumento de textos disponíveis na internet, o desenvolvimento de técnicas de Processamento Computacional da Língua (PCL) tem vindo a aumentar. Os métodos de PCL modernos utilizam frequentemente a tokenização – uma estratégia que divide as palavras em unidades menores – como primeiro passo para a interpretação e processamento de textos.

Neste trabalho, foi criado e analisado um corpus textual em língua portuguesa, baseado nas obras de António Lobo Antunes, em seguida foi implementado um método de tokenização BPE, treinado com um conjunto treino que constitui 90% do corpus inicial, e, por fim, foi comparado com dois métodos de tokenização muito populares: GPT-2 tokenizer (BPE) e BERT tokenizer (WordPiece).

O objetivo do trabalho foi avaliar e comparar a eficácia dos métodos de tokenização utilizados em termos de preservação de informação semântica e capacidade de lidar com palavras fora do vocabulário.

Descrição dos dados recolhidos

O corpus é constituído por excertos das obras de António Lobo Antunes, nomeadamente *Os Cus De Judas* (1979), *Fado Alexandrino* (1983), *Auto Dos Danados* (1985), *A*

Ordem Natural Das Coisas (1992), *O Manual Dos Inquisidores* (1996), *Eu Hei-de Amar Uma Pedra* (2004) e *Até Que As Pedras Se Tornem Mais Leves Que A Água* (2017), sendo todos romances escritos em português. Foram escolhidos excertos representativos do estilo literário do autor, de forma aleatória, obtendo assim, diversidade lexical e estrutural no corpus.

O corpus utilizado encontra-se dividido em duas pastas diferentes: Conjunto de treino, utilizado para treinar o tokenizador BPE desenvolvido neste trabalho; Conjunto de teste, utilizado para avaliar o comportamento do tokenizador referido anteriormente e comparar os resultados obtidos com tokenizadores pré-treinados.

Para a análise do corpus, foi utilizada a biblioteca NLTK, onde se fez a tokenização dos textos, utilizando a divisão dos espaços em branco como separação de tokens, e posteriormente, foi feita a remoção de qualquer pontuação (tais como travessão, vírgula, ponto, ...) para garantir que apenas as palavras fossem consideradas na análise.

Medidas identificadas

Medida	Descrição	Resultado obtido
Número total de documentos	Soma total dos documentos presentes nos conjuntos de treino e de teste	27
Número total de palavras	Soma total de palavras presentes no corpus	6833
Tamanho do vocabulário	Número total de palavras únicas no conjunto de treino	2469
Riqueza lexical	Razão entre o tamanho do vocabulário e o número total de palavras no conjunto de treino	≈ 0.40
Número total de palavras fora do vocabulário (OOV)	Total de palavras presentes no conjunto de teste, mas que não existem no vocabulário	230

```
Número de documentos: 27
Número total de palavras: 6833
Vocabulário: 2469
Riqueza lexical: 0.4021172638436482
Número de palavras OOV: 230
```

Figura 1 - Resultados obtidos referentes às medidas do corpus

Aspetos relevantes identificados

Caraterística	Descrição	Exemplos/Resultado
Palavras mais frequentes	Corresponde às vinte palavras que mais aparecem no corpus. É possível observar que os textos são dominados por stopwords (preposições, artigos e conjunções) e que segue a lei de Zipf (numa vasta amostra de dados, a frequência de um item é inversamente proporcional ao seu ranking)	[('de', 351), ('a', 311), ('e', 226), ('que', 191), ('o', 188), ('do', 110), ('da', 104), ('em', 100), ('os', 97), ('não', 83), ('com', 81), ('um', 72), ('na', 72), ('no', 70), ('as', 66), ('uma', 59), ('se', 57), ('para', 57), ('ao', 49), ('das', 39)]
Exemplos de OOV	Conjunto de 10 palavras fora do vocabulário	['triste', 'demasiado', 'quintais', 'pequena', 'chaminés', 'tabique', 'ensurdecidos', 'recusava', 'faca', 'interior']
Palavras raras	São palavras que aparecem apenas uma vez no corpus. Para exemplificar, foi mostrado 20 das palavras mais raras	['trouxe', 'apareci', 'responder', 'perguntei', 'motivo', 'haver', 'regressado', 'calhar', 'feliz', 'sertões', 'encontrou', 'voltavam', 'máscara', 'boneco', 'garoto', 'silêncios', 'conversas', 'afastavam', 'ouviam', 'tiros']
Palavras longas	Conjunto de 20 palavras com mais de dez caracteres, composto maioritariamente por palavras no plural ou formas verbais pronominais	['recordações', 'continuando', 'devagarinho', 'cemiteriozito', 'consequinte', 'interrompam', 'preocupar-vos', 'encontrei-as', 'choradeiras', 'afastando-se', 'designando-me', 'quarta-feira', 'interessaram', 'consultório', 'incompreensíveis', 'marxismo-leninismo-maoísmo', 'Semelhantes', 'juntavam-se', 'escavacadas', 'metralhadoras']
Estrangeirismos	Representa todas as palavras estrangeiras (com k, w e y) presentes no corpus	['Kelly', 'Kelly', 'Kelly', 'Clark', 'vodka', 'Royal', 'Mickey', 'Royal']
Cobertura	A cobertura representa a proporção de palavras no conjunto de teste que são representadas no vocabulário. Uma percentagem <70% representa uma cobertura baixa, o que pode ser explicado pela riqueza lexical alta (≈40%)	≈0.685

Hápx ratio	O hápx ratio corresponde à proporção de palavras que aparecem apenas uma vez no corpus (hápx legómenon) em relação ao número total de palavras. Uma percentagem alta indica um vocabulário rico e um corpus constituído maioritariamente por textos literários	≈ 0.82
Yule's K	O Yule's K é uma métrica de diversidade lexical que mede a repetição de palavras num corpus. Um Yule's K de 97.4, por exemplo, representa um vocabulário variado	

Top 20 palavras:
 [('de', 351), ('a', 311), ('e', 226), ('que', 191), ('o', 188), ('do', 110), ('da', 104), ('em', 100), ('os', 97), ('não', 83), ('com', 81), ('um', 72), ('na', 72), ('no', 70), ('as', 66), ('uma', 59), ('se', 57), ('para', 57), ('ao', 49), ('das', 39)]

Exemplos OOV:
 ['distante', 'tateante', 'lâmpada', 'vizinho', 'gelatina', 'sombra', 'máximos', 'fugir-nos', 'dignos', 'somente']

Palavras raras (exemplos):
 ['trouxe', 'apareci', 'responder', 'perguntei', 'motivo', 'haver', 'regressado', 'calhar', 'feliz', 'sertões', 'encontrou', 'voltavam', 'máscara', 'boneco', 'garoto', 'silêncios', 'conversas', 'afastavam', 'ouviam', 'tiros']

Figura 2- Resultados obtidos referentes a palavras frequentes, raras e OOV

Palavras longas:
 ['recordações', 'continuando', 'devagarinho', 'cemiteriozito', 'consequinte', 'interrompam', 'preocupar-vos', 'encontrei-as', 'choradeiras', 'afastando-se', 'designando-me', 'quarta-feira', 'interessaram', 'consultório', 'incompreensíveis', 'marxismo-leninismo-maoísmo', 'Semelhantes', 'juntavam-se', 'escavacadas', 'metralhadoras']

Estrangeirismos:
 ['Kelly', 'Kelly', 'Kelly', 'Clark', 'vodka', 'Royal', 'Mickey', 'Royal']

Cobertura: 0.6681096681096681
 Hapax ratio: 0.29620957119859503
 Yule's K: 97.40315547114558

Figura 3 - Resultados obtidos referentes a palavras longas, estrangeirismos, cobertura, hapax ratio e Yule's K

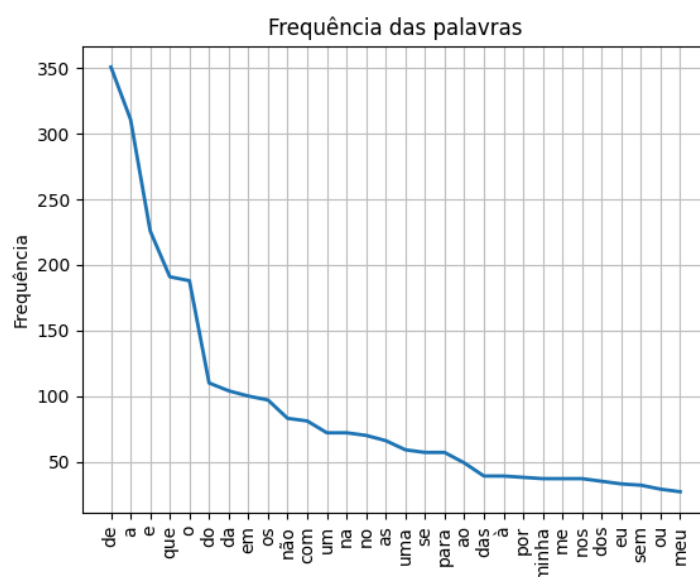


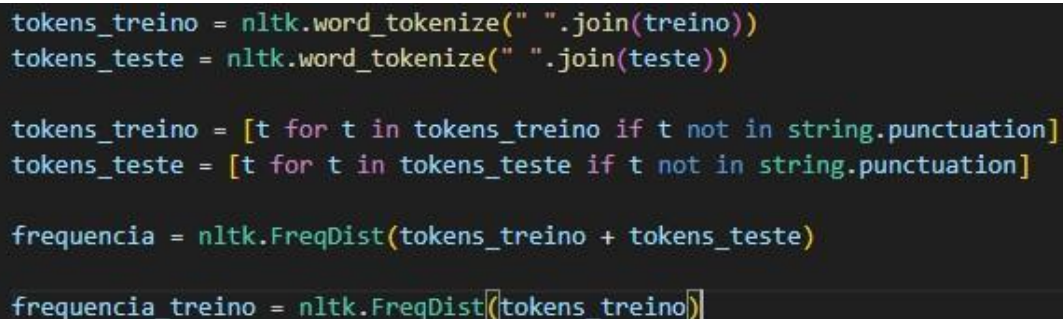
Figura 4 - Gráfico das 30 palavras mais frequentes e demonstração da lei de Zipf

Descrição das Tarefas Desenvolvidas e Procedimentos Efetuados

Análise do corpus

A análise do corpus foi feita no script `analise_corpus.py`, onde os dados do corpus são carregados através da função `carregar_corpus` do script `carregar_corpus.py`, que lê os ficheiros presentes na pasta que lhe é passada como argumento e devolve uma lista de strings, cada uma representando um documento, acabando desta forma com duas variáveis: `treino`, com a lista de textos do conjunto de treino, e `teste`, com a lista de textos do conjunto de teste. A função ainda substitui qualquer quebra de linha (`\n`) por espaço em branco, para não criar conflito na criação de sub-palavras pelos tokenizadores.

De seguida, foi feita a tokenização por espaços em branco utilizando `word_tokenize` do NLTK (Fig. 5) e, por fim, foi retirada a pontuação, pois esta era considerada como uma palavra ou parte de uma, o que causaria erros na análise estatística do corpus. Com as palavras separadas por tokens, é calculada a frequência através do `FreqDist` do NLTK, tanto para o total de palavras, como para as de treino exclusivamente (é necessária a frequência destas para calcular o Yule' K).



```
tokens_treino = nltk.word_tokenize(" ".join(treino))
tokens_teste = nltk.word_tokenize(" ".join(teste))

tokens_treino = [t for t in tokens_treino if t not in string.punctuation]
tokens_teste = [t for t in tokens_teste if t not in string.punctuation]

frequencia = nltk.FreqDist(tokens_treino + tokens_teste)

frequencia_treino = nltk.FreqDist(tokens_treino)
```

Figura 5 - Imagem do código usado para a tokenização com NLTK

Por fim, é calculado todas as medidas e aspetos referidos anteriormente e feito o print destas. Também é utilizada a biblioteca Matplotlib para a criação do gráfico das palavras mais frequentes. Para o uso da biblioteca NLTK é necessário primeiro fazer o download dos recursos `punkt` e `punkt_tab`, que contêm modelos utilizados pelo NLTK para segmentar o texto em palavras e sem estes, o processo de tokenização originará erros durante a execução (ambos os downloads estão comentados no código).

Criação e treino do tokenizador BPE

A criação do BPE foi feita no script `bpe.py`, onde, primeiramente, são carregados os dados do conjunto de treino e, a seguir, é criado e treinado o tokenizador.

Na função `create_tokenizer`, usou-se a biblioteca `tokenizers` para criar um modelo BPE e treiná-lo usando o conjunto de treino e o `trainer` criado dentro da função (Fig. 6), onde é colocado um tamanho máximo de vocabulário de 1500 tokens, para incentivar a criação de sub-palavras, evitando assim a memorização de todas as palavras por inteiro do corpus, e a expressão “<unk>” para qualquer token que seja desconhecido quando o modelo for usado fora do treino.

```
def create_tokenizer(corpus):
    tokenizer = Tokenizer(models.BPE())

    trainer = trainers.BpeTrainer(
        vocab_size=1500,
        special_tokens=["<unk>"]
    )

    tokenizer.train_from_iterator(corpus, trainer)
    tokenizer.save("bpe_tokenizer.json")

    return tokenizer

tokenizer = create_tokenizer(corpus_treino)
```

Figura 6 - Imagem da função que cria e treina o tokenizador BPE

```
Tamanho do vocabulário: 1500

Vocabulário ordenado por id:
[('<unk>', 0), (' ', 1), ('(', 2), (')', 3), (',', 4), ('-', 5), ('.', 6), ('2', 7), ('8', 8), ('9', 9), (':', 10), (';', 11), ('?', 12), ('A', 13), ('B', 14), ('C', 15), ('D', 16), ('E', 17), ('F', 18), ('G', 19)]

Exemplo do vocabulário:
[('ão', 812), ('Nun', 966), ('ul', 200), ('gente ', 998), ('atr', 347), ('os e as ', 1242), ('o cra', 1207), ('arbus', 1091), ('ando-me ', 1310), ('ter ', 1036), ('ós ', 815), ('gr', 243), ('ch', 131), ('filhos ', 1278), ('igo', 918), ('as m', 1099), ('a c', 818), ('no p', 1192), ('As ', 959), ('telefon', 1350)]
```

Figura 7 - Amostra dos 20 primeiros tokens do vocabulário e exemplos de tokens contidos neste

Após a criação do tokenizador, é possível reutilizá-lo através do `Tokenizer.from_file("bpe_tokenizer.json")` que carrega o tokenizador guardado no json `bpe_tokenizer` (essa linha está comentada no código), não sendo necessário a criação repetida deste.

Por fim, foi feito o carregamento do conjunto teste e, através do encode do tokenizador, foi usado o modelo antes criado para tokenizar documento a documento. Para análise do

tokenizador, foi feito print dos primeiros vinte e dois tokens e seus respectivos ids (Fig. 8), onde se pode ler, por exemplo, o excerto “ao apagar-se Lisboa imensa lá fora e o quinto andar inexistente, “, referente ao primeiro documento *eu_hei-de_amar_uma_pedra2.txt*.

```
--- TEXTO ---
ao apagar-se Lisboa imensa lá fora e o quinto andar inexistente, havia as nossas vozes, não haviam
os nós, as luzes de outros prédios, néons, escutávamos os taipais da pastelaria distante que o em
preg

Tokens:
['ao ', 'ap', 'ag', 'ar', '-se ', 'Lisboa ', 'im', 'ens', 'a ', 'lá ', 'for', 'a e ', 'o ', 'quin',
, 'to ', 'and', 'ar ', 'in', 'ex', 'ist', 'ent', 'e, ']

Tokens ids:
[165, 189, 187, 84, 228, 937, 309, 426, 77, 448, 442, 176, 79, 507, 123, 210, 125, 93, 263, 397, 1
17, 330]
```

Figura 8 - Amostra dos 22 primeiros tokens do primeiro documento do conjunto de teste e seus respectivos ids

Utilização de tokenizadores pré-treinados

A utilização e comparação dos tokenizadores pré-treinados foi feita no script `tokenizadores_pre_treinados.py`, no qual foi utilizado o GPT-2 tokenizer (BPE) e o BERT tokenizer (WordPiece).

Para cada documento do conjunto de teste foi feito o print de um excerto do texto e a tokenização feita por ambos os tokenizadores pré-treinados, GPT-2 e BERT (Fig. 9).

No caso do GPT-2, este utiliza o símbolo especial $\tilde{}$ para representar espaços em branco e podemos ver no exemplo abaixo que as letras que são representadas por mais que dois bytes levam a um erro de decodificação (mojibake), neste caso, a letra ‘á’ é tokenizada como $\tilde{A}$ _j. Isto ocorre porque o GPT-2 “quebra” o texto em bytes individuais, mas como a letra ‘á’ é representada por dois bytes diferentes (c3 e A1) ela aparece como $\tilde{A}$ (c3)_j(A1).

No caso do BERT, este utiliza o símbolo ## para indicar dois tokens que juntos formam uma palavra. A escolha para a criação destes grupos de tokens segue a fórmula $score = (freq_of_pair) / (freq_of_first_element \times freq_of_second_element)$, ou seja, o algoritmo prioriza a fusão de pares que são menos frequentes individualmente, o que significa que, dois pares que apareçam sempre um seguido do outro e nunca separados vão ser fundidos através do símbolo ## do algoritmo. No exemplo abaixo também podemos ver que este tokenizador é uncased (‘Lisboa’ passou a ‘lisboa’) e ignora a acentuação (‘lá’ passou a ‘la’).

```

--- TEXTO ---
ao apagar-se Lisboa imensa lá fora e o quinto andar inexistente, havia as nossas vozes, não haviam
os nós, as luzes de outros prédios, néons, escutávamos os taipais da pastelaria distante que o em
preg

BPE tokens:
['ao', 'á', 'aga', '-', 'se', 'áLis', 'boa', 'áim', 'ens', 'a', 'á1', 'Ã', 'áfor', 'a', 'áe',
'á', 'áqu', 'into', 'ánd', 'ar', 'áinex', 'istent', 'e', ',', 'áha', 'via', 'áas', 'á', 'oss',
'as']

WordPiece tokens:
['ao', 'ap', '##aga', '##r', '-', 'se', 'li', '##sb', '##oa', 'im', '##ens', '##a', 'la', 'for', '
##a', 'e', 'o', 'qui', '##nto', 'and', '##ar', 'in', '##ex', '##iste', '##nte', ',', 'ha', '##via',
', 'as', 'nos']

```

Figura 9 - Exemplos da tokenização dos modelos pré-treinados BPE e WordPiece

Conclusões

Em resumo, foi explorado ao longo do trabalho diferentes abordagens de tokenização, nomeadamente GPT-2(BPE), BERT(WordPiece) e um tokenizador BPE criado especificamente para este projeto.

A análise realizada permite concluir que o BPE implementado consegue representar adequadamente o corpus, dividindo as palavras em sub-unidades ou, até mesmo, representá-las por inteiro (como o exemplo da palavra Lisboa na Fig. 8), reduzindo assim o problema de palavras fora do vocabulário, mas devido a ser treinado com um corpus mais reduzido, o seu vocabulário é mais limitado quando comparado com os tokenizadores pré-treinados e, consequentemente, pode vir a ter problemas em tokenizar outros corpora que não este.

Por outro lado, os tokenizadores pré-treinados foram treinados com corpora maiores, o que lhes permite ter um vocabulário mais rico e regras de tokenização mais robustas, conseguindo assim, representar um número maior de palavras e preservar melhor a informação semântica presente nos textos. Todavia, por estes tokenizadores serem treinados com corpora maioritariamente inglês e o corpus utilizado ser em português, ambos falham na tokenização de palavras com acentos, nomeadamente, a palavra 'lá' que nem o GPT-2, nem o BERT conseguiram tokenizar corretamente.

Conclui-se assim que, para este corpus específico, o tokenizador BPE criado, por ter sido treinado com parte do próprio corpus, consegue preservar melhor a informação semântica, mas não apresenta um vocabulário tão diversificado como os tokenizadores pré treinados, o que leva a ser menos eficaz em lidar com palavras foras do vocabulário ou outros corpora de temas diferentes.

Bibliografia

Antunes, A. L. (1983). *Fado Alexandrino*. Lisboa: Dom Quixote.

Antunes, A. L. (2004). *Eu Hei-de Amar uma Pedra*. Lisboa: Dom Quixote.

Antunes, A. L. *A Ordem Natural das Coisas* (texto online).

https://www.deficienciavisual.pt/r-A_ordem_natural_coisas-Lobo_Antunes.htm

Antunes, A. L. *A Ordem Natural das Coisas* disponível no Google Books.

<https://books.google.pt/books?id=0aP6hJ5sq8gC>

Antunes, A. L. *Até que as pedras se tornem mais leves que a água* (excerto).

https://static.publico.pt/files/Ipsilon/2017-11-10/ate_que_as_pedras.pdf

Antunes, A. L. *Excerto de Auto dos Danados*. Fnac-static.

<https://static.fnac-static.com/multimedia/PT/pdf/9789722027151.pdf>

Antunes, A. L. *O Manual dos Inquisidores*(excerto).

<https://static.fnac-static.com/multimedia/PT/pdf/9789722028585.pdf>

Goodreads. *Os Cus de Judas – Quotes*.

<https://www.goodreads.com/work/quotes/1478833-os-cus-de-judas>

Hugging Face. *Tokenization (capítulo 6, LLM Course)*.

<https://huggingface.co/learn/llm-course/en/chapter6/1>

Lamsal, Rabindra. (2025, August 21) *BPE Tokenizer: Training and Tokenization Explained*. Langformers Blog.

<https://blog.langformers.com/bpe-tokenizer-explained/>

NLTK Project. (2025, October 1) *Tokenizing text — NLTK HowTo*.

<https://www.nltk.org/howto/tokenize.html>

Percepolegatto. *Lobo Antunes – O Manual dos Inquisidores*.

<https://www.percepolegatto.com.br/2014/06/19/lobo-antunes-o-manual-dos-inquisidores/>

Pulapakura, Raj (2024, January 24) *BPE Tokenizer Explained* [vídeo]. YouTube.

<https://www.youtube.com/watch?v=BcxJk4WQVIw>